DAA LAB ASSIGNMENT WEEK-4

NAME: KURRA MUNI SANTHAN KUMAR REDDY

ROLL.NO: CH.SC.U4CSE24225

1)Prims Algorithm

CODE:

```
#include <stdio.h>
#define INF 999

int main() {
   int n, cost[10][10], visited[10] = {0};
   int mincost = 0, edges = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter cost adjacency matrix:\n");
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0)
            cost[i][j] = INF;
      }
   }
```

```
visited[0] = 1;

while (edges < n - 1) {
    int min = INF, u = -1, v = -1;

    for (int i = 0; i < n; i++) {
        if (visited[i]) {
            for (int j = 0; j < n; j++) {
                if (!visited[j] && cost[i][j] < min) {
                    min = cost[i][j];
                    u = i;
                    v = j;
                }
            }
        }
    }

    visited[v] = 1;
    mincost += min;
    edges++;

    printf("Edge %d: (%d -> %d) cost = %d\n", edges, u, v, min);
}

printf("Minimum cost = %d\n", mincost);
return 0;
```

}

OUTPUT:

```
santhan@Santhan-PC:~$ ./primsalgorithm
Enter number of vertices: 4
Enter cost adjacency matrix:
0 10 6 5
10 0 0 15
4 3 6 0
7 24 15 6
Edge 1: (0 -> 3) cost = 5
Edge 2: (0 -> 2) cost = 6
Edge 3: (2 -> 1) cost = 3
Minimum cost = 14
```

Interpretation:-

I wrote a C program to find the Minimum Spanning Tree using Prim's Algorithm. In this method, I started with an initial vertex and gradually expanded the tree by adding the minimum cost edge that connects a visited vertex to an unvisited vertex. I used a visited array to keep track of the vertices already included in the spanning tree. In each step, I scanned the adjacency matrix to find the smallest edge connecting the tree to a new vertex and added it to the tree. This process continued until all vertices were connected. Prim's Algorithm has a time complexity of **$O(n^2)$**. Its space complexity is **$O(n)$**.

2)Kruskals Algorithm

CODE:

```
#include <stdio.h>

int parent[10];

int find(int i) {
   while (parent[i])
      i = parent[i];
   return i;
}

int uni(int i, int j) {
   if (i != j) {
      parent[j] = i;
      return 1;
   }
   return 0;
}

int main() {
   int n, cost[10][10], mincost = 0, edges = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);
```

```
printf("Enter cost adjacency matrix:\n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        scanf("%d", &cost[i][j]);
        if (cost[i][j] == 0)
            cost[i][j] = 999;
    }
}

while (edges < n - 1) {
    int min = 999, u = -1, v = -1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (cost[i][j] < min) {
                min = cost[i][j];
                u = i;
                v = j;
            }
        }
    }

    int a = find(u);
    int b = find(v);

    if (uni(a, b)) {
```

```
        printf("Edge %d: (%d -> %d) cost = %d\n", ++edges, u, v, min);
        mincost += min;
      }

      cost[u][v] = cost[v][u] = 999;
    }

    printf("Minimum cost = %d\n", mincost);
    return 0;
}
```

OUTPUT:

```
santhan@Santhan-PC:~$ ./kruskalsalgorithm
Enter number of vertices: 4
Enter cost adjacency matrix:
5 7 3 9
15 7 8 3
0 5 8 0
10 4 89 56
Edge 1: (0 -> 2) cost = 3
Edge 2: (1 -> 3) cost = 3
Edge 3: (2 -> 1) cost = 5
Minimum cost = 11
```

Interpretation:-

I wrote a C program to find the Minimum Spanning Tree using Kruskal's Algorithm. In this code, I considered all the edges in the graph and selected the smallest edge first. I used the union–find method to make sure that adding an edge does not create a cycle. Each time I selected the minimum cost edge, I checked whether the vertices belonged to different sets, and if they did, I added the edge to the spanning tree. This process continued until the tree had exactly n−1 edges. Kruskal's Algorithm has a time complexity of **O(E log E)**, and its space complexity is **O(n)**.